

W6D1 Report – Gioco Zork a scelte multiple in C (con classifica)

Descrizione del lavoro In questo esercizio ci è stato chiesto di creare un gioco in linguaggio C ispirato a Zork, un'avventura testuale famosa. Il gioco doveva contenere 5 domande a scelta multipla, valutare le risposte, calcolare il punteggio e alla fine stampare una classifica con i risultati di più utenti. Tutto doveva funzionare da terminale, senza usare file esterni.

Come ho organizzato il programma

- Ho creato una funzione chiamata faiDomanda() per gestire ogni domanda del quiz.
- Nel main() ho gestito il flusso di gioco, chiedendo il nome, ponendo le domande e salvando i risultati.
- Alla fine viene stampata la classifica con i punteggi di tutti gli utenti che hanno giocato.

Codice del programma

```
#include <stdio.h>
#include <ctype.h>

#define MAX_GIOCATORI 10

int faiDomanda(const char *testo, const char *opzioneA, const char *opzioneB, const char
*opzioneC, char rispostaCorretta, int indice, int testArray[], char risposteCorrette[]) {
    char rispostaUtente;
    printf("\n%s\n", testo);
    printf("A) %s\n", opzioneA);
    printf("B) %s\n", opzioneB);
    printf("C) %s\n", opzioneC);
    printf(">> La tua scelta (A/B/C): ");
    scanf(" %c", &rispostaUtente);
    rispostaUtente = toupper(rispostaUtente);
    risposteCorrette[indice] = rispostaCorretta;

    if (rispostaUtente == toupper(rispostaCorretta)) {
        printf("Risposta corretta!\n");
        testArray[indice] = 1;
        return 1;
    } else {
        printf("Risposta sbagliata! La risposta corretta era %c\n", rispostaCorretta);
        testArray[indice] = 0;
        return 0;
    }
}

int main() {
    char nomi[MAX_GIOCATORI][50];
    int punteggi[MAX_GIOCATORI];
    int numeroGiocatori = 0;
    char giocaAncora;

    do {
        int testArray[5] = {0};
        char risposteCorrette[5];
```

```

int punteggio = 0;

printf("\nInserisci il tuo nome: ");
scanf("%s", nomi[numeroGiocatori]);

printf("\nBenvenuto %s in Zork - L'avventura testuale!\n", nomi[numeroGiocatori]);

    faiDomanda("Ti trovi nella casa bianca. Su un tavolo c'è una lanterna. Cosa devi fare?",
"Ignorare la lanterna", "Prendere la lanterna", "Spegnerla", 'B', 0, testArray, risposteCorrette);
    faiDomanda("Apri la botola e scendi. È buio pesto. Cosa rischi se resti troppo al buio?",
"Niente", "Ti perdi", "Ti mangia un Grue", 'C', 1, testArray, risposteCorrette);
    faiDomanda("Nel labirinto i passaggi sono tutti uguali. Come si esce?", "Andando a caso",
"Mappando i percorsi", "Tornando indietro sempre", 'B', 2, testArray, risposteCorrette);
    faiDomanda("Arrivi a un fiume sotterraneo. Come lo attraversi?", "Nuoti", "Cerchi una barca",
"Salti", 'B', 3, testArray, risposteCorrette);
    faiDomanda("Hai trovato un forziere chiuso. Cosa fai?", "Rompi il forziere", "Usi la chiave
d'ottone", "Lo ignori", 'B', 4, testArray, risposteCorrette);

    for (int i = 0; i < 5; i++) {
        punteggio += testArray[i];
    }

    printf("\n===== REPORT FINALE =====\n");
    printf("%s, hai risposto correttamente a %d su 5 domande.\n", nomi[numeroGiocatori],
punteggio);

    for (int i = 0; i < 5; i++) {
        printf("Stanza %d: ", i + 1);
        if (testArray[i] == 1) {
            printf("Corretta\n");
        } else {
            printf("Sbagliata (giusta: %c)\n", risposteCorrette[i]);
        }
    }

    if (punteggio == 5) {
        printf("Hai conquistato tutto il tesoro di Zork. Complimenti!\n");
    } else if (punteggio >= 3) {
        printf("Hai esplorato bene, ma puoi fare di meglio.\n");
    } else {
        printf("Sei stato sopraffatto dal dungeon. Riprova!\n");
    }

    punteggi[numeroGiocatori] = punteggio;
    numeroGiocatori++;

    printf("\nVuoi far giocare un altro utente? (S/N): ");
    scanf(" %c", &giocaAncora);
    giocaAncora = toupper(giocaAncora);

} while (giocaAncora == 'S' && numeroGiocatori < MAX_GIOCATORI);

```

```

printf("\n===== CLASSIFICA UTENTI =====\n");
for (int i = 0; i < numeroGiocatori; i++) {
    printf("%s - %d/5\n", nomi[i], punteggi[i]);
}

printf("\nGrazie per aver giocato!\n");
return 0;
}

```

Compilazione ed esecuzione su Kali Linux Per compilare e far partire il gioco, ho usato questi comandi:

```

gcc -o zork zork.c
chmod +x zork
./zork

```

Senza il comando `chmod +x zork` non riuscivo a lanciare il file, mi diceva che non esisteva.

Esecuzione del gioco (simulazione completa)

Inserisci il tuo nome: ADR

Ti trovi nella casa bianca. Su un tavolo c'è una lanterna. Cosa devi fare?

- A) Ignorare la lanterna
- B) Prendere la lanterna
- C) Spegnerla

>> B

Risposta corretta!

Apri la botola e scendi. È buio pesto. Cosa rischi se resti troppo al buio?

- A) Niente
- B) Ti perdi
- C) Ti mangia un Grue

>> C

Risposta corretta!

Nel labirinto i passaggi sono tutti uguali. Come si esce?

- A) Andando a caso
- B) Mappando i percorsi
- C) Tornando indietro sempre

>> B

Risposta corretta!

Arrivi a un fiume sotterraneo. Come lo attraversi?

- A) Nuoti
- B) Cerchi una barca
- C) Salti

>> B

Risposta corretta!

Hai trovato un forziere chiuso. Cosa fai?

- A) Rompi il forziere
- B) Usi la chiave d'ottone
- C) Lo ignori

>> B

Risposta corretta!

===== REPORT FINALE =====

ADR, hai risposto correttamente a 5 su 5 domande.

Vuoi far giocare un altro utente? (S/N): S

Inserisci il tuo nome: Valentino

Ti trovi nella casa bianca. Su un tavolo c'è una lanterna. Cosa devi fare?

- A) Ignorare la lanterna
- B) Prendere la lanterna
- C) Spegnerla

>> B

Risposta corretta!

Apri la botola e scendi. È buio pesto. Cosa rischi se resti troppo al buio?

- A) Niente
- B) Ti perdi
- C) Ti mangia un Grue

>> B

Risposta sbagliata! La risposta corretta era C

Nel labirinto i passaggi sono tutti uguali. Come si esce?

- A) Andando a caso
- B) Mappando i percorsi
- C) Tornando indietro sempre

>> A

Risposta sbagliata! La risposta corretta era B

Arrivi a un fiume sotterraneo. Come lo attraversi?

- A) Nuoti
- B) Cerchi una barca
- C) Salti

>> B

Risposta corretta!

Hai trovato un forziere chiuso. Cosa fai?

- A) Rompi il forziere
- B) Usi la chiave d'ottone
- C) Lo ignori

>> C

Risposta sbagliata! La risposta corretta era B

===== REPORT FINALE =====

Valentino, hai risposto correttamente a 2 su 5 domande.

Vuoi far giocare un altro utente? (S/N): S

Inserisci il tuo nome: Sara

Ti trovi nella casa bianca. Su un tavolo c'è una lanterna. Cosa devi fare?

- A) Ignorare la lanterna
- B) Prendere la lanterna
- C) Spegnerla

>> B

Risposta corretta!

Apri la botola e scendi. È buio pesto. Cosa rischi se resti troppo al buio?

- A) Niente
- B) Ti perdi
- C) Ti mangia un Grue

>> C

Risposta corretta!

Nel labirinto i passaggi sono tutti uguali. Come si esce?

- A) Andando a caso
- B) Mappando i percorsi

C) Tornando indietro sempre

>> B

Risposta corretta!

Arrivi a un fiume sotterraneo. Come lo attraversi?

A) Nuoti

B) Cerchi una barca

C) Salti

>> C

Risposta sbagliata! La risposta corretta era B

Hai trovato un forziere chiuso. Cosa fai?

A) Rompi il forziere

B) Usi la chiave d'ottone

C) Lo ignori

>> B

Risposta corretta!

===== REPORT FINALE =====

Sara, hai risposto correttamente a 4 su 5 domande.

Vuoi far giocare un altro utente? (S/N): N

===== CLASSIFICA UTENTI =====

ADR - 5/5

Sara - 4/5

Valentino - 2/5

Tabella dei comandi usati nel programma e cosa fanno

Comando	A cosa serve
#include <stdio.h>	Mi serve per usare printf e scanf, cioè per stampare e leggere input da tastiera
#include <ctype.h>	Serve per usare toupper, così le lettere vengono sempre maiuscole e non ci sono problemi nei controlli
#define MAX_GIOCATORI 10	Imposto una costante, così limito il numero massimo di giocatori
char	Tipo di variabile per salvare lettere, tipo A, B o C
int	Tipo di variabile per salvare numeri interi, tipo punteggi
scanf(" %c", &variabile)	Legge da tastiera una lettera, lo spazio serve per evitare problemi con invio
toupper()	Trasforma una lettera minuscola in maiuscola
if / else	Serve per controllare se una cosa è vera o falsa e fare qualcosa di diverso
== / !=	Controllo se due cose sono uguali o diverse
for	Serve per ripetere più volte una cosa, tipo per ogni domanda
return 0;	Finisce il programma senza errori
[]	Li uso per dichiarare e usare array, cioè liste di valori (tipo risposte e punteggi)
printf()	Stampa messaggi a schermo
gcc	Compila il file .c
chmod +x nome	Rende il file eseguibile, sennò da errore
./nome	Fa partire il programma compilato

Conclusione Durante questo esercizio ho notato tante cose utili: per prima cosa, quanto è importante l'organizzazione del codice, soprattutto quando si iniziano ad avere più utenti e si vogliono salvare i punteggi in modo ordinato.

Ho imparato anche come gestire bene i cicli `for`, come confrontare i caratteri indipendentemente da maiuscole o minuscole con `toupper()`, e soprattutto ho visto come usare array per salvare

risposte e punteggi. Una cosa fondamentale è stato capire l'uso del `chmod +x` perché senza quello il programma non partiva da terminale.

La parte della classifica finale è stata quella che mi ha fatto riflettere di più, perché ho dovuto pensare a come memorizzare più nomi e punteggi senza usare file. Infine, con questo gioco ho avuto modo di usare praticamente tutti i comandi e concetti che abbiamo visto nelle slide: variabili, array, condizioni `if`, cicli `for`, funzioni, librerie standard e anche la parte terminale di Linux. È stato utile anche per fare un po' di debugging quando qualcosa non funzionava.